PYTHON PROGRAMMING
C0_02 ASSIGNMENT

NAME: SHABANA THASLEEM A.W

COURSE: PYTHON PROGRAMMING

COURSE CODE: CSA0802
ASSIGNMENT: 2
NATURE OF THE ASSIGNMENT: Individual

TOTAL MARKS: 50

SUBMISSION DATE: 12/11/26

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

#1. LEARNING OUTCOMES/CO2 COVERAGE

```python
num = int(input("Enter a number: "))

if num > 0:
    print("Positive number")
elif num < 0:
    print("Negative number")
else:
    print("Zero")
```

```
Enter a number: 10
Positive number
```

```python
print("For Loop")
for i in range(1, 6):
    print(i)

print("While Loop")
i = 1
while i <= 5:
    print(i)
    i += 1
```

```
For Loop
1
2
3
4
5
While Loop
1
2
3
4
5
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
print("Break:")
for i in range(1, 6):
    if i == 3:
        break
    print(i)

print("Continue:")
for i in range(1, 6):
    if i == 3:
        continue
    print(i)

print("Pass:")
for i in range(1, 6):
    if i == 3:
        pass
    print(i)
```

```
Break:
1
2
Continue:
1
2
4
5
Pass:
1
2
3
4
5
```

```python
for i in range(1, 6):
    print(i)
else:
    print("Loop completed")
```

```
1
2
3
4
5
Loop completed
```

```python
def add(a, b):
    return a + b

x = int(input("Enter first number: "))
y = int(input("Enter second number: "))

result = add(x, y)
print("Sum =", result)
```

```
Enter first number: 10
Enter second number: 20
Sum = 30
```

```python
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

num = int(input("Enter a number: "))
print("Factorial =", factorial(num))
```

```
Enter a number: 5
Factorial = 120
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
text = input("Enter a string: ")

print("Length:", len(text))
print("Uppercase:", text.upper())
print("Lowercase:", text.lower())
print("First character:", text[0])
print("Last character:", text[-1])
print("Slicing:", text[0:3])
```

```
Enter a string: python
Length: 6
Uppercase: PYTHON
Lowercase: python
First character: p
Last character: n
Slicing: pyt
```

```python
text = input("Enter a string: ")

new_text = "J" + text[1:]

print("Original string:", text)
print("Modified string:", new_text)
```

```
Enter a string: Hello
Original string: Hello
Modified string: Jello
```

```python
def add(a, b):
    return a + b

x = int(input("Enter first number: "))
y = int(input("Enter second number: "))

result = add(x, y)
print("Sum =", result)
```

```
Enter first number: 10
Enter second number: 20
Sum = 30
```

#2. GENERAL INSTRUCTIONS

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
import time

n = int(input("Enter a number: "))

start = time.perf_counter()

total = 0
for i in range(1, n + 1):
    total += i

end = time.perf_counter()

print("Sum using for loop:", total)
print("For loop time:", end - start)

start = time.perf_counter()

total = 0
i = 1
while i <= n:
    total += i
    i += 1

end = time.perf_counter()

print("Sum using while loop:", total)
```

```
Enter a number: 5
Sum using for loop: 15
For loop time: 8.430004527326673e-06
Sum using while loop: 15
While loop time: 3.460001607891172e-06
Optimized sum: 15
Optimized time: 1.1399970389902592e-06
```

```python
print("BREAK STATEMENT")

for i in range(1, 6):
    if i == 3:
        break
    print(i)

print("CONTINUE STATEMENT")

for i in range(1, 6):
    if i == 3:
        continue
    print(i)

print("PASS STATEMENT")

for i in range(1, 6):
    if i == 3:
        pass
    print(i)
```

```
BREAK STATEMENT
1
2
CONTINUE STATEMENT
1
2
4
5
PASS STATEMENT
1
2
3
4
5
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
n = int(input("Enter a number: "))

if n <= 1:
    print("Not a prime number")
else:
    for i in range(2, n):
        if n % i == 0:
            print("Not a prime number")
            break
    else:
        print("Prime number")
```

```
Enter a number: 7
Prime number
```

```python
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    return a / b

x = int(input("Enter first number: "))
y = int(input("Enter second number: "))

print("Addition:", add(x, y))
print("Subtraction:", subtract(x, y))
print("Multiplication:", multiply(x, y))

if y != 0:
    print("Division:", divide(x, y))
else:
    print("Division not possible")
```

```
Enter first number: 20
Enter second number: 10
Addition: 30
Subtraction: 10
Multiplication: 200
Division: 2.0
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

#4. ASSIGNMENT QUESTIONS

```python
units = int(input("Enter electricity units: "))
ration = input("Do you have a ration card? (yes/no): ").lower()
age = int(input("Enter age of household member: "))

if units <= 100:
    print("Full Subsidy")

if units <= 200 and units >= 101 and ration == "yes":
    print("Partial Subsidy")

if units > 200 and age > 80:
    print("Special Subsidy")

if units > 200 and age <= 80:
    print("No Subsidy")

if units >= 101 and units <= 200 and ration == "no":
    print("No Subsidy")
```

```
Enter electricity units: 100
Do you have a ration card? (yes/no): yes
Enter age of household member: 45
Full Subsidy
```

```python
units = int(input("Enter electricity units: "))
ration = input("Do you have a ration card? (yes/no): ").lower()
age = int(input("Enter age of household member: "))

if units <= 100:
    print("Full Subsidy")

elif 101 <= units <= 200 and ration == "yes":
    print("Partial Subsidy")

elif units > 200 and age > 80:
    print("Special Subsidy")

else:
    print("No Subsidy")
```

```
Enter electricity units: 250
Do you have a ration card? (yes/no): no
Enter age of household member: 85
Special Subsidy
```

```python
units = 100
ration = "yes"

if units <= 100:
    print("Full Subsidy")

if units <= 200 and ration == "yes":
    print("Partial Subsidy")
```

```
Full Subsidy
Partial Subsidy
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
units = int(input("Enter units: "))
ration = input("Ration card (yes/no): ").lower()
age = int(input("Enter age: "))

match True:
    case _ if units <= 100:
        print("Full Subsidy")
    case _ if 101 <= units <= 200 and ration == "yes":
        print("Partial Subsidy")
    case _ if units > 200 and age > 80:
        print("Special Subsidy")
    case _:
        print("No Subsidy")
```

```
Enter units: 400
Ration card (yes/no): yes
Enter age: 18
No Subsidy
```

```python
correct_otp = "1234"
attempts = 0

while attempts < 3:

    otp = input("Enter OTP: ")

    if otp.strip() == "":
        print("Blank entry! Please enter OTP.")
        continue

    attempts += 1

    if otp == correct_otp:
        print("OTP Verified Successfully!")
        break
    else:
        print("Incorrect OTP!")
        print("Attempts remaining:", 3 - attempts)

else:
```

```
Enter OTP: 1111
Incorrect OTP!
Attempts remaining: 2
Enter OTP:
Blank entry! Please enter OTP.
Enter OTP: 2222
Incorrect OTP!
Attempts remaining: 1
Enter OTP: 1234
OTP Verified Successfully!
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
correct_otp = "1234"

for i in range(3):

    while True:
        otp = input("Enter OTP: ")

        if otp.strip() == "":
            print("Blank entry! Please enter OTP.")
            continue

        break

    if otp == correct_otp:
        print("OTP Verified Successfully!")
        break

    else:
        print("Incorrect OTP!")
        print("Attempts remaining:", 2 - i)

else:
```

```
Enter OTP: 1111
Incorrect OTP!
Attempts remaining: 2
Enter OTP: 2222
Incorrect OTP!
Attempts remaining: 1
Enter OTP: 1234
OTP Verified Successfully!
```

```python
for i in range(3):
    print("Checking OTP")

else:
    print("Account Locked for 30 Minutes!")
```

```
Checking OTP
Checking OTP
Checking OTP
Account Locked for 30 Minutes!
```

```python
# Vehicle Registration Number Validator

def normalise(text):
    # TODO 1: Remove spaces and hyphens
    # Convert to uppercase

    text = text.replace(" ", "")
    text = text.replace("-", "")
    text = text.upper()

    return text


def is_valid(reg):
    # TODO 2: Check length
    if len(reg) != 9 and len(reg) != 10:
        return False

    # Check state letters
    if not reg[0:2].isalpha():
        return False
```

```
Enter vehicle registration number: tn 09 ab 1234
Valid Registration Number
Receipt Number: TN 09 AB 1234
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
text = "hello"
new_text = text.upper()

print(text)
print(new_text)-
```

```
hello
HELLO
```

```python
import time
from io import StringIO

# Create items
items = [f"Item{i}:Rs{i % 50}" for i in range(200000)]

# Method 1: Using + operator
def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ";"
    return text

# Method 2: Using join()
def build_join(parts):
    return ";".join(parts) + ";"

# Method 3: Using StringIO
def build_stringio(parts):
    output = StringIO()

    for p in parts:
```

```
Same output: True
Length: 3048890
+ time: 0.0505 seconds
Join time: 0.0056 seconds
StringIO time: 0.0312 seconds
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```
def average_time(function, parts):
    total = 0

    for i in range(3):
        start = time.perf_counter()
        function(parts)
        total += time.perf_counter() - start

    return total / 3


print("Average + time:",
      average_time(build_plus, items))

print("Average join time:",
      average_time(build_join, items))

print("Average StringIO time:",
      average_time(build_stringio, items))
```

```
# Incorrect program

def add_item(item, order=[]):
    order.append(item)
    return order


print("Customer 1:", add_item("Burger"))
print("Customer 2:", add_item("Pizza"))
print("Customer 3:", add_item("Juice"))
```

```
Customer 1: ['Burger']
Customer 2: ['Burger', 'Pizza']
Customer 3: ['Burger', 'Pizza', 'Juice']
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
# Correct program

def add_item(item, order=None):

    if order is None:
        order = []

    order.append(item)

    return order


# Customer 1
customer1 = add_item("Burger")
print("Customer 1:", customer1)

# Customer 2
customer2 = add_item("Pizza")
print("Customer 2:", customer2)

# Customer 3
customer3 = add_item("Juice")
```

```
Customer 1: ['Burger']
Customer 2: ['Pizza']
Customer 3: ['Juice']
```

```python
def student(name, age):
    print("Name:", name)
    print("Age:", age)

student(age=20, name="Arun")
```

```
Name: Arun
Age: 20
```

```python
def bill(item, price=50):
    print(item, price)

bill("Tea")
bill("Coffee", 30)
```

```
Tea 50
Coffee 30
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
def calculate_bill(price, gst):
    total = price + gst
    return price, gst, total


a, b, c = calculate_bill(100, 18)

print("Price:", a)
print("GST:", b)
print("Total:", c)
```

```
Price: 100
GST: 18
Total: 118
```

```python
def display():
    x = 10
    print("Local:", x)

display()
```

```
Local: 10
```

```python
x = 100

def display():
    print("Global:", x)

display()
```

```
Global: 100
```

```python
def outer():
    x = 10

    def inner():
        nonlocal x
        x = 20

    inner()
    print("Value:", x)

outer()
```

```
Value: 20
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
x = 10

def change():
    global x
    x = 50

change()

print(x)
```

```
50
```

```python
# College Canteen Order System

def add_item(item, order=None):

    if order is None:
        order = []

    order.append(item)

    return order


def calculate_total(price, gst=5):

    total = price + (price * gst / 100)

    return price, gst, total


# Customer 1
order1 = add_item("Burger")
order1 = add_item("Juice", order1)
```

```
Customer 1 Order: ['Burger', 'Juice']
Customer 2 Order: ['Pizza', 'Tea']
Customer 3 Order: ['Sandwich']
Price: 200
GST: 5 %
Total Bill: 210.0
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
# Incorrect function

def add_item_buggy(item, order=[]):
    order.append(item)
    return order


# Correct function

def add_item(item, order=None):

    if order is None:
        order = []

    order.append(item)
    return order


print(add_item_buggy("Tea"), add_item_buggy("Vada"))

print(add_item("Tea"), add_item("Vada"))

# Print default values
print("Default values:", add_item_buggy.__defaults__)
```

```
['Tea', 'Vada'] ['Tea', 'Vada']
['Tea'] ['Vada']
Default values: (['Tea', 'Vada'],)
```

```python
def add_item(item, order):
    order.append(item)
    return order


customer1 = []
customer2 = []

print(add_item("Tea", customer1))
print(add_item("Vada", customer2))
```

```
['Tea']
['Vada']
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
def create_order():

    order = []

    def add_item(item):
        order.append(item)
        return order

    return add_item


customer1 = create_order()
customer2 = create_order()

print(customer1("Tea"))
print(customer1("Vada"))

print(customer2("Coffee"))
```

```
['Tea']
['Tea', 'Vada']
['Coffee']
```

```python
# Plain recursive method

calls = 0

def ways_naive(n):

    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_naive(n-1) + ways_naive(n-2)


n = int(input("Enter number of steps: "))

calls = 0

result = ways_naive(n)

print("Number of ways:", result)
print("Function calls:", calls)
```

```
Enter number of steps: 5
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
from functools import lru_cache

calls = 0

@lru_cache(maxsize=None)
def ways_memo(n):

    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_memo(n-1) + ways_memo(n-2)


n = int(input("Enter number of steps: "))

calls = 0

result = ways_memo(n)

print("Number of ways:", result)
print("Function calls:", calls)
```

```
Enter number of steps: 8
Number of ways: 34
Function calls: 9
```

```python
from functools import lru_cache

calls = 0

def ways_naive(n):
    """Ways to climb using 1 or 2 steps."""

    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_naive(n-1) + ways_naive(n-2)


@lru_cache(maxsize=None)
def ways_memo(n):

    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_memo(n-1) + ways_memo(n-2)
```

```
naive: 121393 calls = 242785
memo : 121393 calls = 26
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
def ways_loop(n):

    a = 1
    b = 1

    for i in range(n):
        a, b = b, a + b

    return a


n = int(input("Enter number of steps: "))

print("Number of ways:", ways_loop(n))
```

```
Enter number of steps: 5
Number of ways: 8
```

```python
def ways_optimized(n):

    a = 1
    b = 1

    for i in range(n):
        a, b = b, a + b

    return a


n = int(input("Enter number of steps: "))

print("Optimized result:", ways_optimized(n))
```

```
Enter number of steps: 5
Optimized result: 8
```

PYTHON PROGRAMMING
C0_02 ASSIGNMENT

```python
import time

# Original recursive program

def ways_naive(n):

    if n <= 1:
        return 1

    return ways_naive(n-1) + ways_naive(n-2)


# Redesigned iterative program

def ways_optimized(n):

    a = 1
    b = 1

    for i in range(n):
        a, b = b, a + b

    return a


# Compare both programs
```

```
Steps: 10
Original result: 89
Optimized result: 89
Same result: True
Original time: 1.1200005246791989e-05
Optimized time: 2.5500048650428653e-06

Steps: 20
Original result: 10946
Optimized result: 10946
Same result: True
Original time: 0.0011799200001405552
Optimized time: 3.3800024539232254e-06

Steps: 25
Original result: 121393
Optimized result: 121393
Same result: True
Original time: 0.015910558999166824
Optimized time: 5.410001904238015e-06

Steps: 30
Original result: 1346269
Optimized result: 1346269
Same result: True
Original time: 0.16279735800344497
```

FINAL COMPARISON:

Feature	Original recursion	Redesigned loop
Time complexity	$O(2^n)$	O(n)
Extra space	O(n)	O(1)
Repeated calculations	Yes	No
Recursion error	Possible	No
Suitable for 1000 steps	No	Yes